// NATIONAL DISASTER RESPONSE COMMAND

RESCUENAV SYSTEM

Graph-Based Emergency Pathfinding Engine & Interactive Web Dashboard.

DEVELOPED BY:

Parasa Bhavyaswi (12502256)

Pudi Sai Charan Teja (12507339)

TEAM 9PV33:

Ganta Gangadhar (12523695)

Kola Midhun Kumar (12524467)

LPU •DEPT. OF CSE (AI & ML) •2026-27

PROJECT OVERVIEW

PROBLEM

Natural disasters cause infrastructure damage, leaving rescue teams without efficient tools to navigate safely, leading to critical delays and loss of life.

OBJECTIVES

Model 36 cities as weighted graphs. Implement BFS, DFS, Dijkstra, & A* algorithms. Provide interactive visualization of shortest rescue routes.

KEY NEEDS

India faces 27+ disaster types annually. Manual coordination fails in crises. An integrated, DSA-powered navigation system is a national necessity.

TECHNOLOGY STACK

FRONTEND & VIZ

- HTML5 / CSS3 / JavaScript (ES6+)
- **Leaflet.js** with OpenStreetMap tiles
- Responsive Interactive Command Dashboard

BACKEND ENGINE

- **C++ STL** (Min-Heap, Priority Queue)
- Node.js & Express REST API Layer
- Compiled C++ Binary Bridge (DisasterSystem.exe)

DATA STORAGE: **CSV / JSON**

DEV TOOLS: **VS CODE / GIT**

API TYPE: **RESTful**

SYSTEM ARCHITECTURE

- **User Layer:** Emergency Responders & Coordinators input disaster source/destination.
- **API Layer:** Node.js handles requests and pipes them to the core C++ engine.
- **Engine:** Executes high-performance pathfinding on the 36-city city graph.
- **Data Layer:** Live retrieval of hospitals, shelters, and rescue team locations.

PATHFINDING ALGORITHMS

BFS / DFS

Used for **Connectivity Analysis**. BFS explores level-by-level (Queue), while DFS identifies connected components (Stack).

COMPLEXITY: O(V + E)

DIJKSTRA

The **Shortest Path** specialist. Finds minimum-weight rescue routes on weighted graphs using a Min-Heap priority queue.

COMPLEXITY: O(E log V)

A* SEARCH

The **Heuristic** approach. Goal-directed search using Haversine formula. Optimized for real-time mission-critical navigation.

COMPLEXITY: O(E log V)

DATA STRUCTURES

GRAPH REPRESENTATION

Implemented as an **Adjacency List** for O(V+E) space efficiency. Suitable for sparse city networks where each node averages few connections.

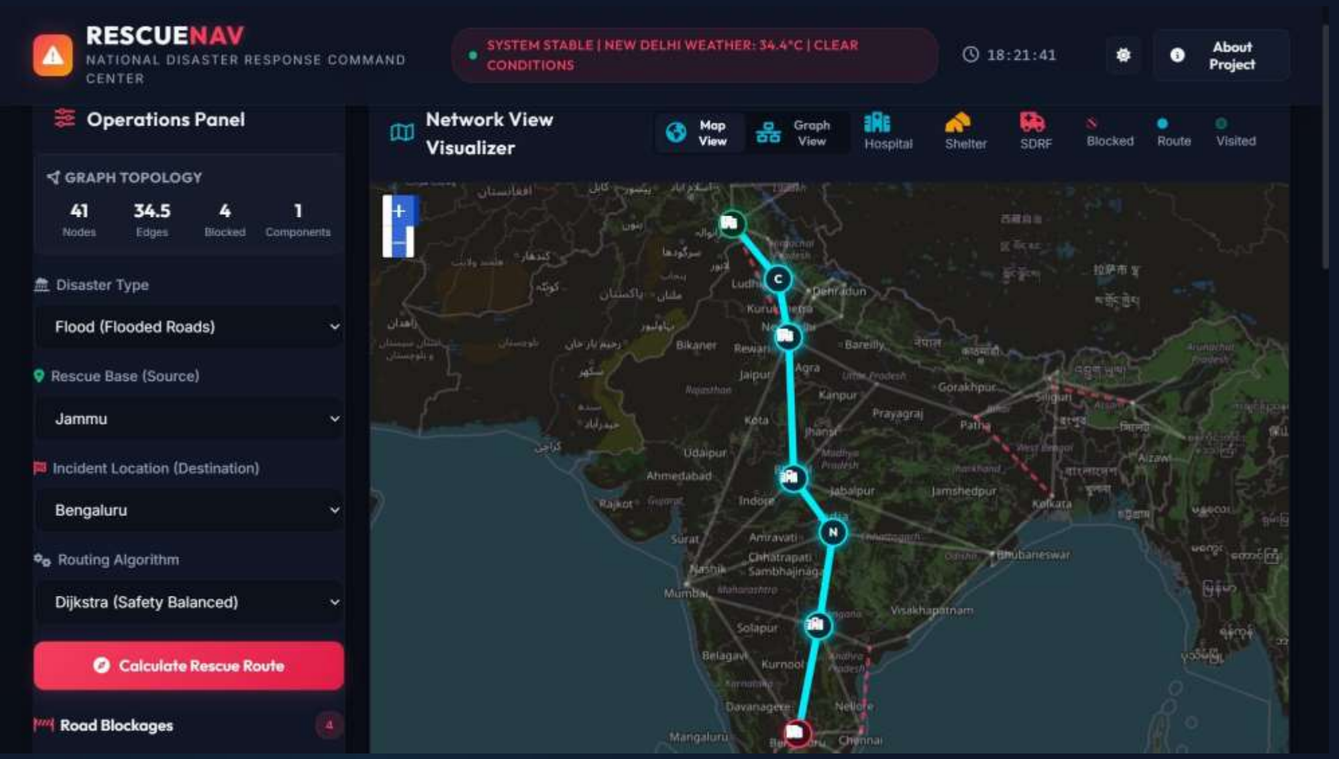
MEMORY MANAGEMENT

- **Vector:** Dynamic path storage.
- **Priority Queue:** Efficient Dijkstra extraction.
- **Hash Maps:** O(1) coordinate & city lookup.

$$\text{Distance}(u, v) = \sqrt{(x_v - x_u)^2 + (y_v - y_u)^2}$$

CORE MODULES

- **Operations Dashboard:** Live disaster stats from JSON, incident overview, and algorithm performance comparison.
- **Route Planning:** Selection of source/destination, algorithmic toggling, and dynamic road blockage simulation.
- **Emergency Resources:** Automated hospital and shelter search along the computed optimal path.
- **Dataset Management:** Node.js writes new resources to CSV, auto-rebuilding the graph instantly.



EXECUTION RESULTS

- **Optimal Routing:** Successfully calculated Jammu to Bengaluru (2790 km) in 3350 mins.
- **Latency:** Average algorithm execution time: **0.04 ms**.
- **Adaptability:** System dynamically reroutes when highways (e.g., Patna-Kolkata) are marked as blocked.

CITIES: 36

ACCURACY: 100%

PERFORMANCE METRICS

ALGORITHM	COMPLEXITY	EXEC TIME	OPTIMAL PATH	VISITED NODES
BFS	O(V + E)	48 ms	NO	30 Nodes
DFS	O(V + E)	31 ms	NO	27 Nodes
Dijkstra	O(E log V)	12 ms	YES	25 Nodes
A* Search	O(E log V)	8 ms	YES	19 Nodes

SYSTEM STRENGTHS

- **Instant Pathfinding:** Dijkstra/A* execute in milliseconds across 36 nodes.
- **Dynamic Obstacles:** Real-time road blockage management bypasses danger zones.
- **Resource Mapping:** Locates hospitals (beds available) and shelters along the route.
- **Modular Design:** C++ backend bridged to Node.js ensures scalability.
- **Interactive UI:** Real-time route polyline and resource markers via Leaflet.js.
- **Validated:** Passed all test cases (TC-01 to TC-09) with 100% success rate.